# OOB_shift_reaper y NAMU - revisión de fuente v1.91.0

Ciclo de vida, propiedad y límites actuales de implementación

## 1. Dos fronteras de ciclo de vida

OOB_shift_reaper rige el reconocimiento de larga duración durante una ejecución común de la aplicación. NAMU es la limpieza prioritaria del reconocimiento en el path de apagado deAgentConfig.ready(). Son fronteras distintas: una ventana de gracia durante una ejecución no da permiso de sobrevivir a la aplicación que lo posee.

Nmapper y Discoverer transmiten la salida del subproceso. Su runner considera que un proceso vivo podría estar colgado después de la gracia de arranque y de hang_detect_idle_seconds sin salida. Antes del umbral de ejecución libre registra una advertencia y continúa. Después de ese umbral, un proceso que sigue colgado se mata por árbol y se conserva la salida parcial. Un proceso que ya salió se drena y se reporta como terminado. Un proceso que está produciendo salida no se mata sólo porque este umbral de OOB en particular se haya cumplido.

La condición es elapsed > OOB_shift_reaper, evaluada por el loop de sondeo. “Exactamente el segundo 3601” es una ilustración, no una garantía de agendado. La cancelación explícita del usuario, las fallas del sistema operativo o el apagado de la aplicación siguen siendo causas independientes de terminación.

# 2. Comportamiento y configuración específicos por agent

Nmapper y Discoverer usan _run_cmd_oob en sus propios módulos de agent. El OOB_shift_reaper predeterminado es de 3600 segundos y el hang_detect_idle_seconds predeterminado es 120. Un intervalo sin salida es una heurística operativa, no una prueba de que un escaneo esté en deadlock. La gracia de arranque y la cadencia de logging le pertenecen al runner. Los comandos de setup/bootstrap/meta pueden conservar sus propios timeouts más cortos. Los límites de escáner que especifica el usuario siguen siendo relevantes.

Kalier es diferente: su solicitud espera por HTTP a un servicio remoto de Kali. No puede transmitir ese escaneo remoto a través de estos runners locales. La fuente usa OOB_shift_reaper como el presupuesto de espera de HTTP, con 3600 como predeterminado. Los timeouts del servicio remoto y el tiempo de vida del proceso remoto los controla el host de Kali. Subir la espera local no sube un límite del servidor remoto, y cerrar un cliente HTTP local no prueba que la ejecución remota se haya detenido.

Ubicaciones principales en la fuente: agents/nmapper/nmapper.py::_run_cmd_oob; agents/discoverer/discoverer.py::_run_cmd_oob; agents/kalier/kalier.py. Usa nombres de función en vez de números de línea históricos, que cambian entre releases.

# 3. Limpieza de apagado: alcance y límites

apps.py::namu_kill_recon_gods corre primero en cleanup_pool_on_shutdown. Elige la raíz del pool según el modo source o frozen, enumera los procesos locales, coincide los paths del pool y los nombres de nmapper/discoverer/kalier, e invoca recursive_kill. Intenta primero los descendientes y luego los padres, tolerando procesos que desaparecieron y accesos denegados. La limpieza más amplia también maneja otros procesos rastreados.

La prioridad es incondicional respecto a la ventana de tiempo transcurrido del OOB. No es una garantía absoluta de limpieza inmediata: la enumeración de procesos, los permisos, las condiciones de carrera y la terminación forzada de la aplicación pueden impedir que el código normal de limpieza termine. En particular, la limpieza local no puede certificar que un servicio remoto de Kali haya matado su escaneo. La redacción universal “cada hijo al instante” del diseño anterior describía la intención con más fuerza de la que la implementación puede probar.

Este documento revisa el comportamiento de la fuente. No afirma una nueva prueba de estrés de apagado, una prueba de cancelación remota ni una aceptación de paquete frozen.

# 4. Relación con Terminater e Interactor

Terminater es dueño de sesiones de terminal reales a través de TerminalHub. Los reapers consultan terminal_hub.live_pids(); cerrar una página del browser no mata por sí mismo las shells. Config → Live Editor & Terminater da el ritmo por cuenta, el Enter manual y la narración acotada. Un timeout de espera de comando no mata el proceso. El tiempo de vida de la terminal y sus propios controles de comando-colgado/toma-de-control-del-usuario están separados de la ventana de gracia de OOB del reconocimiento.

Interactor es un módulo, no un proceso de agent ni una terminal. Opera el editor en vivo a través de un puente de browser autenticado. Lee el texto sin guardar, escribe segmentos cortos de enseñanza y le entrega los campos al usuario. El input del usuario, Detener y la desconexión interrumpen la escritura conservando el prefijo; las pestañas ocultas se pausan con un periodo de recuperación acotado. No ejecuta el código editado, y editar no guarda un archivo de manera implícita.

El Guardar como nativo del canvas escribe a un destino local que elige el usuario, fuera de Downloads, usando verificaciones de revisión y capacidades ligadas a la cuenta/sesión. Estos mecanismos del editor no cambian NAMU ni el backup/restauración de la base de datos.

Contratos relacionados: docs/terminater.md, docs/interactor.md, docs/canvas-editing.md y docs/changes/2026-10-23-v1.91.0.md. Los dossiers completos de arquitectura y de resumen describen la aplicación más amplia y distinguen la evidencia de la fuente de las pruebas instaladas.